



## 1. Ce este un sir

Un vector de `char` care se termina cu caracterul `'\0'`. Pozitiile incep de la 0.

|   |   |   |   |   |   |   |    |
|---|---|---|---|---|---|---|----|
| A | n | a |   | a | r | e | \0 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7  |

```
#include <cstring>

char s[100] = "Ana are";
cin >> s;           // citeste pana la primul spatiu
cin.getline(s, 100); // citeste toata linia

for (int i = 0; s[i] != '\0'; i++)
    cout << s[i];
```

## 2. Caracterele sunt numere (codul ASCII)

| Ce verifici / faci        | Cum                                             |
|---------------------------|-------------------------------------------------|
| litera mica               | <code>c &gt;= 'a' &amp;&amp; c &lt;= 'z'</code> |
| litera mare               | <code>c &gt;= 'A' &amp;&amp; c &lt;= 'Z'</code> |
| cifra                     | <code>c &gt;= '0' &amp;&amp; c &lt;= '9'</code> |
| mica → mare / mare → mica | <code>c - 32 / c + 32</code>                    |
| cifra → numar             | <code>c - '0' ( '7' - '0' = 7)</code>           |
| vocala                    | <code>strchr("aeiouAEIOU", c) != NULL</code>    |

## 3. Functiile principale

Exemplele pornesc de la `char s[100] = "Ana are mere";`

| Funcția                     | Ce face                                                                      | Exemplu → rezultat                                                                    |
|-----------------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <code>strlen(s)</code>      | lungimea (fara <code>'\0'</code> )                                           | <code>strlen(s) → 12</code>                                                           |
| <code>strcpy(d, s)</code>   | copiază <code>s</code> în <code>d</code>                                     | <code>strcpy(d, "ana") → d = ana</code>                                               |
| <code>strcat(d, s)</code>   | lipește <code>s</code> la finalul lui <code>d</code>                         | <code>strcat(d, " mere") → ana mere</code>                                            |
| <code>strcmp(a, b)</code>   | compara alfabetic:<br><code>&lt;0, 0 sau &gt;0</code>                        | <code>strcmp("ana", "are") → &lt;0</code><br><code>strcmp("ana", "ana") → 0</code>    |
| <code>strchr(s, c)</code>   | prima aparitie a caracterului;<br><code>NULL</code> daca lipseste            | <code>strchr(s, 'r') → re mere</code><br>pozitia: <code>strchr(s, 'r') - s → 5</code> |
| <code>strrchr(s, c)</code>  | ultima aparitie a caracterului                                               | <code>strrchr(s, 'a') → are mere</code>                                               |
| <code>strstr(s, t)</code>   | prima aparitie a sirului <code>t</code> ;<br><code>NULL</code> daca lipseste | <code>strstr(s, "re") → re mere</code>                                                |
| <code>strtok(s, sep)</code> | imparte sirul in cuvinte                                                     | vezi mai jos                                                                          |

## 4. Separarea in cuvinte cu strtok

```
char s[100] = "Ana are mere";
char *p = strtok(s, " ");      // primul apel: cu s
while (p != NULL) {
    cout << p << endl;        // Ana / are / mere
    p = strtok(NULL, " ");     // urmatoarele: cu NULL
}
```

### Atentie

- `s1 = s2` si `s1 == s2` nu merg la siruri — folosesti `strcpy` si `strcmp`.
- Vectorul trebuie sa aiba loc si pentru `'\0'`.
- `strtok` strica sirul original — daca iti trebuie, fa-i o copie cu `strcpy` inainte.